Greenwash

Design Artifact

SENG 401 L02 — Group 02

March 2026

1. Architectural Style: Layered Architecture

Greenwash is built using a layered architecture with five distinct layers. Each layer has a single well-defined responsibility and communicates only with the layer directly below it. There are no upward dependencies, meaning the business logic layer has no knowledge of HTTP, WebSockets, or any transport mechanism. This strict layering enables independent testing, parallel development, and clean separation between the game's rules, its network protocol, and its visual presentation.

1.1 Layer Descriptions

Presentation Layer (TypeScript + Phaser 3 + Vite): The frontend is a single-page Phaser 3 game with 11 scenes managed by Phaser's scene manager. The entry point (main.ts) registers all scenes, and the primary orchestrator is BoardGameScene. Rather than placing all logic in one monolithic scene, BoardGameScene delegates to a set of specialized modules in the frontend/game/ directory: GameState.ts holds all mutable game state as a shared object passed by reference; GameSocketHandler.ts manages the WebSocket connection, message routing, and deferred message queuing during animations; HUDBuilder.ts builds the top player card HUD and phase bar; PolicyHolderBuilder.ts renders the sustainable/exploitative policy holder panels with clickable slots; DrawPileBuilder.ts renders the card draw pile; ElectionTrackerBuilder.ts visualizes the failed election counter; OverlayManager.ts wires overlay scene events (role acknowledged, character selected, vote cast) to WebSocket messages; and PhaseAnnouncer.ts displays full-screen phase transition overlays. The api.ts module provides the REST client (createLobby, joinLobby, leaveLobby, getLobbyPlayers) and WebSocket client (createGameSocket). Cross-platform compatibility is achieved through HTML5 Canvas/WebGL rendering via Phaser 3, working on both desktop and mobile browsers.

Application Layer (Python + FastAPI + Uvicorn): The backend exposes two communication interfaces through a single FastAPI application (main.py). REST endpoints are defined in lobby.py (POST /lobby/create, /lobby/join, /lobby/leave; GET /lobby/{code}/players) and results.py (GET /results/). The WebSocket handler (game.py) accepts persistent connections at /ws/{lobby_code}/{username} and routes 10 distinct message types (start_game, nominate_vice, cast_vote, leader_discard, vice_enact, next_round_ready, reset_game, character_selected, role_acknowledged) to dedicated handler functions. The lobbies.py state manager maintains LobbyState dataclass instances in memory, each holding a player set, active WebSocket connections, ready-player tracking, and an optional GameState reference. Player membership in lobbies is persisted to MySQL so it survives reconnections, while transient game state lives in memory for performance. The application registers CORS middleware for cross-origin requests and seeds the database on startup via the data layer.

Business Logic Layer: This layer contains all game rules and is entirely free of I/O dependencies. game_engine.py defines the GameState class, which implements an 8-phase state machine (LOBBY, ROLE_REVEAL, NOMINATION, ELECTION, LEADER_DISCARD, VICE_ENACT, RESOLUTION, GAME_OVER). It manages the full Leader/Vice legislative flow: the leader nominates a vice (with term limits preventing re-nomination of the previous leader or vice), all players vote by strict majority, the leader draws 3 cards and discards 1, the vice enacts 1 of the remaining 2, and win conditions are checked (5 sustainable = Reformers win, 3 exploitative = Exploiters win). An election tracker forces the top card after 3 consecutive

failed elections. Supporting modules are similarly pure: roles.py assigns secret roles using a configurable distribution table (e.g., 5 players = 2 exploiters), deck.py creates PolicyCard objects from the database (with a fallback to placeholder cards), and voting.py resolves votes by strict majority. Because this layer has zero I/O, all 53 unit tests run in under 2 seconds without any network or database setup.

Data Access Layer: This layer abstracts all database operations behind simple function calls. database.py manages a MySQL connection pool (MySQLConnectionPool with pool_size=5), configured via environment variables (MYSQLHOST, MYSQLPORT, MYSQLUSER, MYSQLPASSWORD, MYSQLDATABASE). models.py provides save_game_result(winner) and get_recent_results(limit) for game outcome persistence. repository.py provides get_all_cards() which reads policy card content from the cards table. seed.py runs initialize_and_seed_database() on server startup, creating tables if needed and inserting 17 policy cards (11 exploitative, 6 sustainable) with educational content about greenwashing.

External Systems: MySQL 8 (green_db) serves as the persistent data store with 6 tables (lobbies, lobby_players, games, game_players, cards, users), enforcing referential integrity through foreign key constraints with ON DELETE CASCADE. Docker Compose orchestrates three containers (Python backend, Node frontend, MySQL 8.4 database) for local development. Railway provides cloud hosting for production deployment.

1.2 Justification for Layered Architecture

Separation of game logic from transport: The game engine has zero knowledge of WebSockets, HTTP, or any network protocol. The same GameState class is called identically whether it's driven by WebSocket messages, REST endpoints, or direct function calls in unit tests. This is the single most important architectural decision: it means we can verify all game rules through fast, deterministic tests without mocking any I/O.

Parallel team development: With 6 team members, the layered structure allowed simultaneous work. Frontend developers built Phaser scenes and the game/ module system using the documented WebSocket message types as a contract. Backend developers implemented the game engine and API. Database work (schema, seeds, connection pooling) proceeded independently. The layers only interact through well-defined interfaces.

Frontend modularity via composition: Within the presentation layer, the game/ directory uses a composition pattern where BoardGameScene acts as an orchestrator, delegating UI construction to specialized builder modules. This prevents the main scene from becoming a monolithic file and allows each visual component (HUD, policy holders, draw pile, election tracker, phase announcer) to be developed and maintained independently.

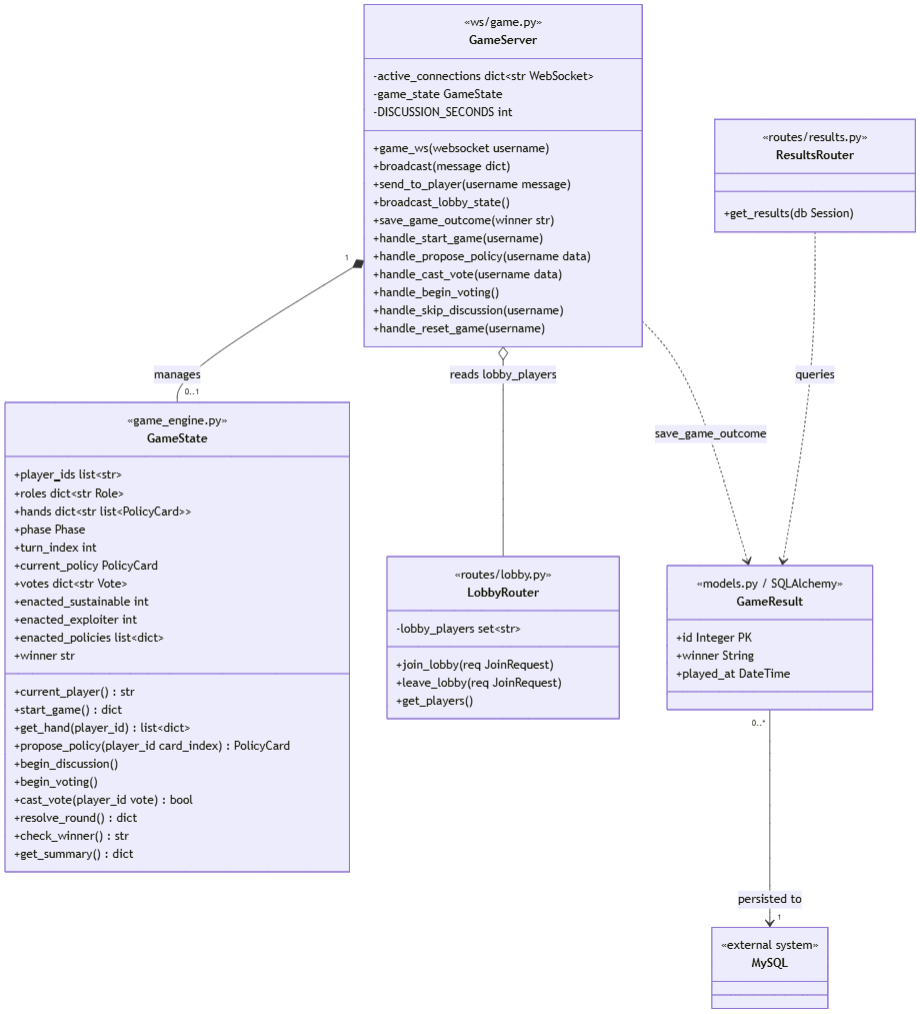
Appropriate scale: For a real-time multiplayer game with 5-10 players per lobby, a single FastAPI process handling both REST and WebSocket connections is sufficient. A microservice architecture would add unnecessary complexity. The layered approach provides the structure needed for maintainability without the operational overhead of distributed systems.

2. Use Case Diagram

Figure 1: Use Case Diagram

3. Class Diagram

The class diagram shows the backend system's object structure. The central class is GameState (game_engine.py), which holds all mutable game data and implements the phase state machine with methods for each game action. LobbyState (lobbies.py) is a dataclass holding a player set, WebSocket connections, and an optional GameState reference (0..1 composition). The GameServer (game.py) manages WebSocket connections and routes messages to handler functions that call GameState methods. Four enumerations define the game's state space: Phase (8 values), Role (Reformer/Exploiter), Vote (Approve/Reject), and PolicyType (Sustainable/Exploitative). The data layer classes (GameResult, Database, Repository) handle MySQL persistence through pooled connections.



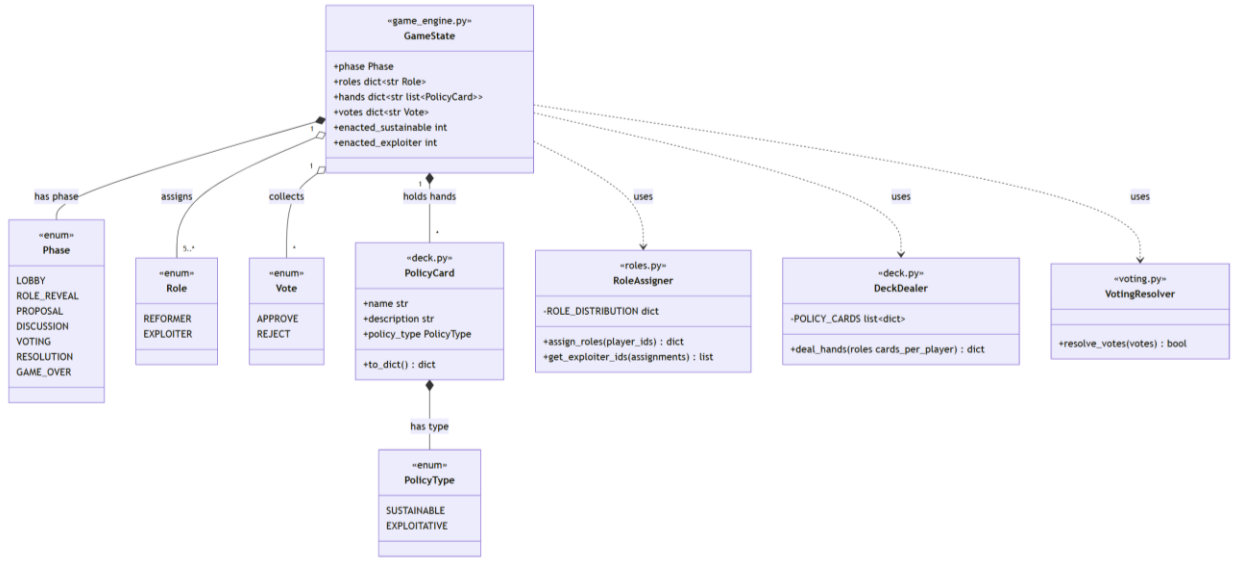


Figure 2: UML Class Diagram

4. Sequence Diagram

The sequence diagram traces one complete game round through all five phases: Nomination (Leader nominates a Vice), Election (all players vote), Leader Discard (Leader draws 3 cards, discards 1), Vice Enact (Vice picks 1 of 2), and Resolution (policy applied, win check, database write if game over). Five actors participate: the Leader Client, All Players (Clients), the GameServer (WebSocket handler), the GameEngine (game_engine.py), and the Database (MySQL). Note that the Leader and Vice receive private card information that other players do not see, and the database is only contacted if a win condition is met.

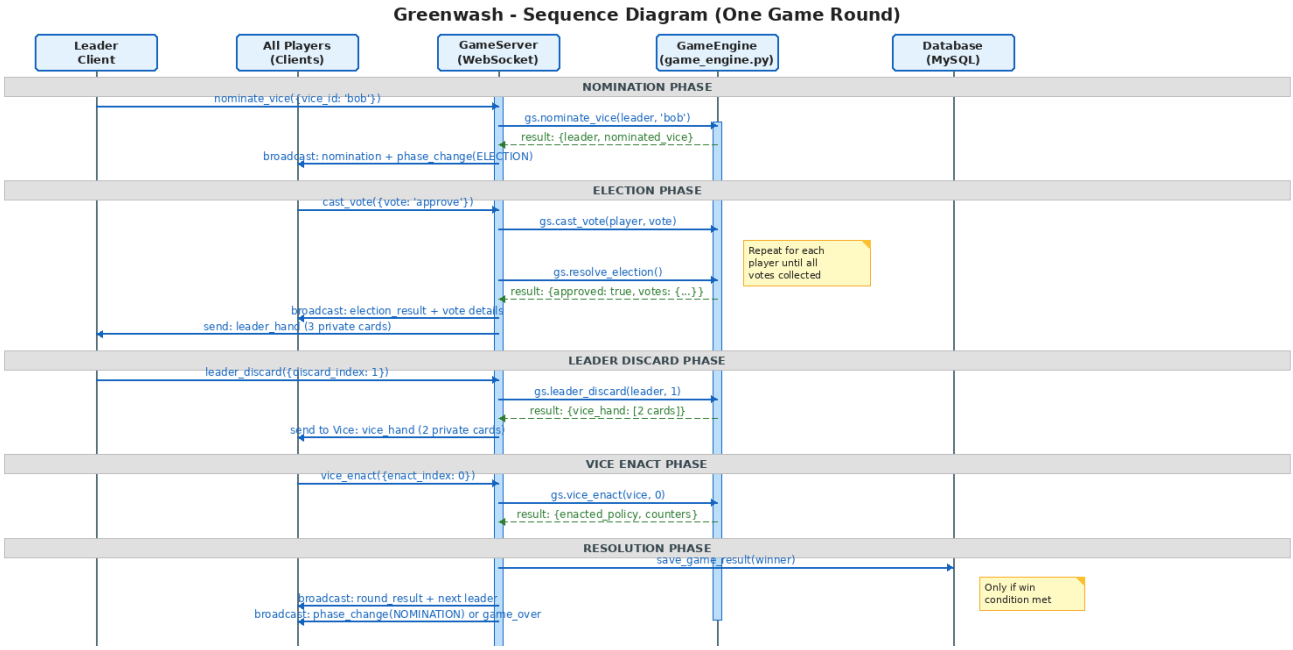


Figure 3: Sequence Diagram (One Game Round)

5. Activity Diagram

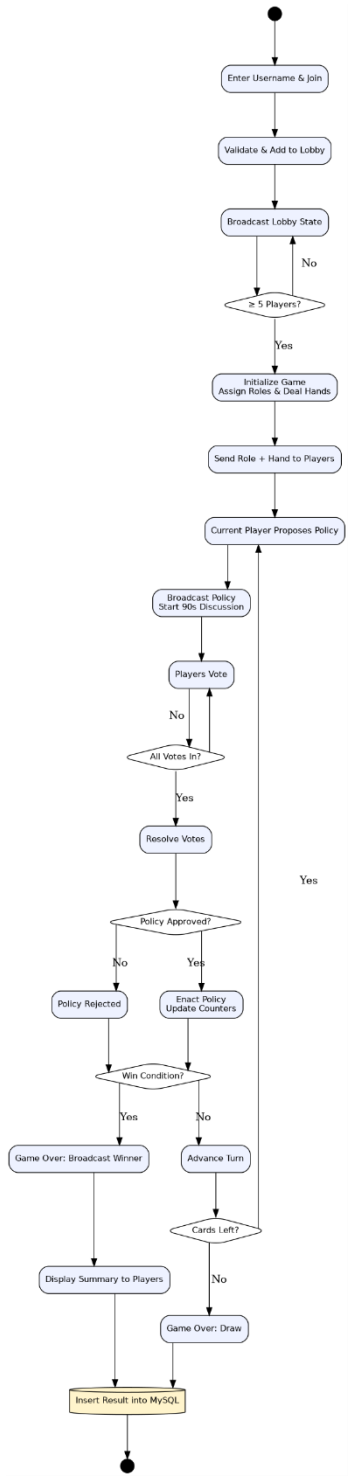


Figure 4: Activity Diagram

5. Database Schema

The MySQL database (green_db) consists of six tables:

lobbies: Stores active lobby sessions with a unique 6-character lobby code (CHAR(6) UNIQUE) and creation timestamp. Primary key: lobby_id (auto-increment).

lobby_players: Tracks which players are in which lobby. Foreign key to lobbies(lobby_id) with ON DELETE CASCADE. Unique constraint on (lobby_id, username) prevents duplicate player entries. Includes joined_at timestamp for ordering.

games: Records completed game sessions with status (ENUM: active/completed/cancelled), winner name, and game_date timestamp.

game_players: Links players to games with per-game statistics: rounds played, score, and is_winner boolean. Foreign keys to both games(game_id) and users(user_id) with CASCADE deletes. Unique constraint on (game_id, user_id).

users: Player profiles with username (UNIQUE), password, and aggregate statistics: total_games_played, total_wins, total_losses. Creation timestamp tracked.

cards: Contains the 17 policy cards seeded on startup. Each card has card_name, card_type (sustainable or exploitative), card_detail (the front text players see), and hover (the educational context text that reveals real-world greenwashing information). This table is the source of truth for all policy card content and connects the gameplay to SDG 12, 13, and 16 through its educational content.

Referential integrity is enforced through foreign key constraints with ON DELETE CASCADE, ensuring that removing a lobby automatically removes its associated players, and removing a game removes its player associations. The database connection is managed through a MySQLConnectionPool with a pool size of 5, configured via environment variables to support both local development and cloud deployment.